Feedback Incorporation Ledger

Student	Christopher Graham
Student number	ST10440190
Module	PROG7312 Programming 3B
Purpose	The updates made to Smart-X in response to marker feedback on Parts 1 and 2, as required by the Part 3 brief

Each entry records the feedback received, what was actually wrong, what changed, and where to find the change.

Part 2 feedback and the response

Part 2 scored 38 out of 100. Investigating the feedback showed a single root cause behind four of the five criteria, which is addressed first because the rest depends on it.

F1. Root cause: the Part 2 code was never pushed to GitHub

> **Marker feedback (Stacks and Queues, 4/15):** "A queue supports path validation, not the required telemetry ingestion and critical-alert processing. No command undo stack found."
>
> **Marker feedback (Sets, 3/10):** "No relevant `HashSet<T>`, `ISet<T>` or `SortedSet<T>` implementation found."
>
> **Marker feedback (Predictive Engine, 5/30):** "The required engine and its query-history-driven recommendations are absent."

What was actually wrong. The feedback was accurate about what had been delivered. The Part 2 implementation existed and ran, but across 41 commits that were never pushed. The branches on GitHub still held the Part 1 code. The only `Queue` visible to the marker was `DeviceNode.IsValidPath`, a Part 1 helper that validates a deployment path, which is precisely what the feedback describes.

The cause was a branch per feature workflow used without a merge step: five feature branches, none merged into `master`, and the working folder sitting on a documentation branch that contained none of the implementation. Everything ran locally, which is what kept the gap invisible.

What changed.

1. Every feature branch was merged and pushed. `master` now contains the full implementation.
2. A clean clone test was adopted as the last step before any submission: clone the repository into a new folder, run the API, the dashboard and the simulator from that clone, and confirm the application works. Nothing is treated as delivered until it passes from a fresh clone.
3. The same check was applied to Part 3 before writing this document.

Where to see it. `master` on https://github.com/BIGchrxs/SmartX.Core.PROG7312, which now holds `IngestionQueue.cs`, `CommandService.cs`, `AlertTracker.cs`, `InsightEngine.cs`, `TelemetryHistory.cs`, the simulator and the full dashboard.

F2. Sorted dictionaries were reported as absent

> **Marker feedback (Dictionaries, 10/15):** "Dictionary-based sensor registration and lookup exist, but the required sorted timelines are absent."

What was actually wrong. `TelemetryHistory` uses a `SortedDictionary<DateTime, HistoryPoint>` per device and always had, but it was in an unpushed commit. Beyond that, the sorted timeline was only visible as a drawn chart, with nothing on screen naming the structure or showing that the readings were ordered and queryable by range.

What changed.

1. The sorted dictionary is on `master` (see F1).
2. A **Device lookup** panel was added to the Command Stream page. Typing a device ID in any case shows the matched device, how many readings are stored out of the 500 kept, and the oldest and newest timestamps. This makes both dictionaries visible: the case insensitive registry lookup and the ordered history behind it.
3. Part 3 adds a second, deeper ordered structure for the same data: an AVL tree per device in `TelemetryArchive`, which answers explicit `from` and `to` range queries over thousands of readings and reports what each query cost.

Where to see it. `smart-x-dashboard/src/components/DeviceLookup.tsx`, `SmartX.Api/Services/TelemetryHistory.cs`, `SmartX.Api/Structures/AvlTree.cs`, and README Test 4 and Test 8.

F3. The set based alert tracking left no visible evidence

> **Marker feedback (Sets, 3/10):** "No relevant `HashSet<T>`, `ISet<T>` or `SortedSet<T>` implementation found."

What was actually wrong. `AlertTracker` uses a `HashSet<string>` of `deviceId:AlertType` keys to make duplicate alerts impossible. Besides being unpushed, its effect was invisible by design: the whole point of the set is that nothing happens on a repeat, so the interface showed one alert and gave no sign that two others had been refused.

What changed.

1. `AlertTracker` now counts every repeat the set refuses, in `DuplicatesIgnored`.
2. An **Alert history** panel on the Command Stream page shows **Alerts raised** next to **Repeats ignored by the set**, with the log of Raised and Recovered entries beneath it. Running the `power-spike` scenario, which sends three critical readings, raises one alert and ignores two.
3. Part 3 adds further set usage with visible results: `MeshNetwork` uses a `HashSet` as the visited guard in both traversals and as the membership test in Prim's algorithm, and a `HashSet` of powered down nodes drives the failure simulation.

Where to see it. `SmartX.Api/Services/AlertTracker.cs`, `smart-x-dashboard/src/components/AlertHistory.tsx`, and README Test 5.

F4. Navigation and the pillar that was not finished

> **Marker feedback (Main Menu Navigation, 16/30):** the band awarded covers "Navigation menu is present, but switching views drops state or introduces noticeable UI freezing. Some of Part 1 work is included or does not work perfectly."

What was actually wrong. The hash router, the shared state provider and the kept page state were all in unpushed commits, so the marker saw the Part 1 interface. Separately, the third pillar card was genuinely disabled, and the Part 3 rubric asks for asynchronous navigation specifically.

What changed.

1. The full navigation is on `master` (see F1): a hash router with real links, all pages kept mounted, shared state above the pages, per page scroll memory and values kept in session storage.
2. The third pillar is now live. All three menu cards open a working page.
3. Navigation was made genuinely asynchronous. Each pillar is a separate JavaScript bundle fetched the first time it is opened. To satisfy the Part 2 requirement at the same time, a page that has been opened stays mounted and is only hidden, so asynchronous loading never costs any state.

Where to see it. `smart-x-dashboard/src/App.tsx`, `components/Landing.tsx`, `components/NavBar.tsx`, and README Test 12. The build output shows one chunk per pillar.

F5. The suggestion engine

> **Marker feedback (Predictive Engine, 5/30):** "The required engine and its query-history-driven recommendations are absent."

What was actually wrong. `InsightEngine` existed, unpushed. It ranks insights with a priority queue and produces four kinds: critical alerts, trend predictions from a least squares fit, habits learned from past operator actions, and frequently repeated searches.

What changed.

1. The engine is on `master` (see F1).
2. The README gained an explicit test for it, including how to build the search and command history the engine learns from, so the query history driven behaviour can be reproduced rather than taken on trust.

Where to see it. `SmartX.Api/Services/InsightEngine.cs`, `SmartX.Api/Services/ActivityLog.cs`, and README Test 6.

Part 1 feedback and brief requirements revisited

While reviewing Parts 1 and 2 against the brief, two requirements were found to be partly met.

F6. MAC addresses were named but never implemented

What was wrong. The Part 1 brief asks for “Device MAC Address / Unique Identifier” under Sensor Payload Management, and the Part 2 rubric refers to “O(1) device lookups via MAC addresses”. Part 3 requires tracking routing paths “using their unique MAC addresses or identifiers”. Smart-X had a `DeviceId` with a comment claiming it was the MAC address, and no actual hardware address anywhere.

What changed.

1. `SensorRegistration` gained a real `MacAddress` property.
2. `MacAddressPolicy` validates the format, normalises it, and generates a stable address in Espressif’s registered block (`24:6F:28:...`) for a device that does not supply one, derived from the device ID so it does not change between restarts.
3. `SensorRegistry` keeps a second dictionary mapping address to device ID, giving genuine O(1) lookup by MAC, and rejects an address already claimed by another device.
4. `GET /sensors/by-mac/{mac}` was added, and the Part 3 routing panel accepts either a device ID or a MAC address.

Where to see it. `SmartX.Api/Services/MacAddressPolicy.cs`, `SmartX.Api/Services/SensorRegistry.cs`, `SmartX.Api/Program.cs`, and README Test 11.

F7. Readings were displayed without units

What was wrong. The live feed and the sensor cards showed bare numbers. A reading of 24 could be degrees, percent or watts, which makes a monitoring interface ambiguous at exactly the moment it matters.

What changed. A `unitFor` helper derives the unit from the zone and category, matching the backend’s own threshold policy, and units now appear in the live feed, on the sensor cards, in the device lookup, on the timeline axis and in the Part 3 historical log.

Where to see it. `smart-x-dashboard/src/status.ts` and the components that call `unitFor`.

Defects found and fixed during this work

These were found by testing rather than by marker feedback, and are listed for completeness.

Defect	Cause	Fix
A command to an actuator would revert about a second later	The simulator read all actuator states once per cycle, so a command landing mid cycle was overwritten by a stale echo	Each actuator is now read immediately before it reports, as real hardware does
A warning read “above the warning limit” when the value equalled the limit	The comparison used `>=`, the message did not say so	Message changed to “at or above the limit”
API error messages appeared in the interface wrapped in JSON quotes	The error body is sometimes a JSON string	`unwrap` in `api.ts` now parses a JSON string before displaying it
Merge conflicts on `SmartX.Api.AssemblyInfo.cs` and similar	Generated `bin` and `obj` output had been committed	`.gitignore` covers `bin/`, `obj/`, `node_modules/`, `dist/` and `uploads/`
Horizontal scrolling on a phone sized screen	A row of buttons could not wrap	`.row-actions` now wraps
The mesh map left a large empty band above the graph and ran off the bottom of the screen	The drawing canvas was nearly square while the panel it sits in is wide	The canvas is now wide, and the nodes are laid out left to right by how far their zone sits from the gateway
Node names overlapped the power cost printed on each link	Every label was drawn directly above its node, whatever was already there	Each label is placed on the side where its node has fewest links, worked out from the directions of the links themselves
Every working node looked the same, whatever its readings said	The map coloured nodes by whether the mesh could reach them, and nothing else	Shape now carries the role, colour carries sensor health, and mesh failures are drawn as shapes so the two never collide

---

Summary of changes made for this submission

Area	Change
Delivery	All work merged and pushed to `master`; clean clone test adopted before submission
Sets	Refused duplicate count added and surfaced in a new Alert history panel
Dictionaries	New Device lookup panel showing case insensitive lookup and ordered history
Trees	AVL archive with range queries, plus a built in comparison against an unbalanced tree
Graphs	Mesh graph with BFS, DFS, Prim, Kruskal, routing and failure simulation
MAC addresses	Implemented properly, with an O(1) index and a lookup route
Navigation	Third pillar enabled; navigation made asynchronous without losing state
Units	Readings now display their unit everywhere
Topology readability	Mesh redrawn on a wide canvas so it fits on one screen, sensors coloured by health, live values on the map, labels placed clear of the links
Packaging	Docker Compose added, so the API, dashboard and simulator start with one command
Documentation	README extended with Part 3, data structure explanations, how to read the map, the Docker setup, and Tests 8 to 12

Christopher Graham | ST10440190